Practical Work 4: Word Count with MapReduce

Le Quy Ton
23BI14425

December 15, 2025

1 Introduction

The objective of this practical work is to implement the classic "Word Count" problem using the MapReduce programming model. We aim to understand the flow of data processing including Mapping, Shuffling, and Reducing.

2 Implementation Choice

We chose a **Python-based Streaming approach** combined with Linux standard utilities.

- **Why Python?** It allows for rapid development of the Mapper and Reducer logic with clean syntax for string manipulation.
- **Why Linux Pipes?** Following the "Invent yourself" guideline, we utilized the Unix philosophy. Instead of setting up a heavy Hadoop cluster, we simulate the distributed nature using the pipeline:

  ```
  cat input.txt | ./mapper.py | sort | ./reducer.py
  ```

  This effectively demonstrates the MapReduce architecture where `sort` handles the crucial "Shuffle and Sort" phase.

3 System Design and Operation

3.1 Mapper Logic

The Mapper reads text line by line. It tokenizes the text into words and emits a key-value pair for each word: $< word, 1 >$.

3.2 Reducer Logic

The Reducer receives sorted input (grouped by keys). It iterates through the lines, summing up the counts for identical consecutive keys. When a new key appears, it outputs the total for the previous key.

3.3 Process Visualization

Figure 1 illustrates how our code processes the input "three witches watch three".

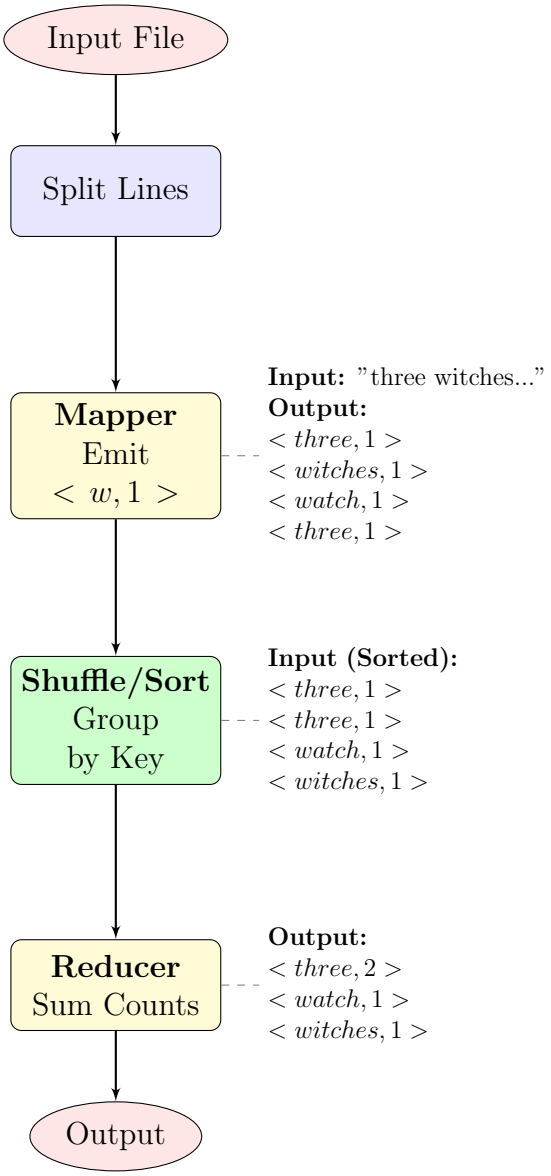

Figure 1: Workflow of the implemented Word Count MapReduce

4 Execution and Results

We verified the implementation by running it on a Linux environment (Debian). We created a dummy input file `input.txt` containing repeated words and executed the pipeline.

Figure 2 shows the terminal output, confirming that the system correctly counts the occurrences of each word (e.g., "bon" appears 4 times, "ba" appears 2 times).

```
tonle@cust-debian:~/Test$ chmod +x mapper.py reducer.py
tonle@cust-debian:~/Test$ echo "ba ba be be lon ton bon bon ton bon bon" > input.txt
tonle@cust-debian:~/Test$ cat input.txt | ./mapper.py | sort | ./reducer.py
ba      2
be      2
bon     4
lon     1
ton     2
tonle@cust-debian:~/Test$ cat ./input.txt
ba ba be be lon ton bon bon ton bon bon
```

Figure 2: Successful execution of MapReduce Word Count

5 Code Implementation

5.1 Mapper (mapper.py)

```
import sys
for line in sys.stdin:
    words = line.strip().split()
    for word in words:
        print(f"{word}\t1")
```

5.2 Reducer (reducer.py)

```
import sys
curr_word = None
curr_count = 0

for line in sys.stdin:
    word, count = line.strip().split('\t', 1)
    count = int(count)
    if curr_word == word:
        curr_count += count
    else:
        if curr_word:
            print(f"{curr_word}\t{curr_count}")
        curr_word = word
        curr_count = count
if curr_word:
    print(f"{curr_word}\t{curr_count}")
```

6 Conclusion

In this practical work, we successfully implemented a Word Count application using the MapReduce paradigm without relying on complex frameworks like Hadoop or Spark. By utilizing Python for logic processing and Linux pipes for data streaming and sorting, we gained a clear understanding of the underlying mechanics of MapReduce.

The experiment demonstrated that the core efficiency of MapReduce lies in the "Shuffle and Sort" phase, which groups related data together, allowing the Reducer to process data sequentially. This lightweight implementation serves as a strong foundation for understanding how large-scale distributed systems process massive datasets.